

19 Těžké problémy

Ohlédněme se za předchozími kapitolami: pokaždé, když jsme potkali nějakou úlohu, dovedli jsme ji vyřešit algoritmem s polynomiální časovou složitostí, tedy $\mathcal{O}(n^k)$ pro pevné k . V prvním přiblížení můžeme říci, že polynomialita docela dobře vystihuje praktickou použitelnost algoritmu.⁽¹⁾

Existují tedy polynomiální algoritmy pro všechny úlohy? Zajisté ne – jsou dokonce i takové úlohy, jež nelze vyřešit žádným algoritmem. Ale i mezi těmi algoritmicky řešitelnými se běžně stává, že nejlepší známé algoritmy jsou exponenciální, nebo dokonce horší.

Pojďme si pár takových příkladů předvést. Navíc uvidíme, že ačkoliv je neumíme efektivně řešit, jde mezi nimi nalézt zajímavé vztahy a pomocí nich obtížnost problémů vzájemně porovnávat. Z těchto úvah vyrůstá i skutečná teorie složitosti se svými hierarchiemi složitostních tříd. Následující kapitolu tedy můžete považovat za malou ochutnávku toho, jak se teorie složitosti buduje. Čtenářům toužícím po hlubším porozumění doporučujeme knihu Arory a Baraka [1].

19.1 Problémy a převody

Aby se nám teorie příliš nerozkošatila, omezíme své úvahy na rozhodovací problémy. To jsou úlohy, jejichž výstupem je jediný bit – máme rozhodnout, zda vstup má či nemá určitou vlastnost. Vstup přitom budeme reprezentovat řetězcem nul a jedniček – libovolnou jinou „rozumnou“ reprezentaci dokážeme na binární řetězec převést v polynomiálním čase. Za velikost vstupu budeme vždy považovat délku řetězce.

Nyní tuto představu přetavíme do exaktní definice:

Definice: *Rozhodovací problém* (zkráceně *problém*) je funkce z množiny $\{0, 1\}^*$ všech řetězců nad binární abecedou do množiny $\{0, 1\}$.

Ekvivalentně bychom se na problém mohli také dívat jako na nějakou množinu $A \subseteq \{0, 1\}^*$ vstupů, na něž je odpověď 1. Tento přístup mají rádi v teorii automatů.

Příklad problému: *Bipartitní párování* – je dán bipartitní graf a číslo $k \in \mathbb{N}$. Máme odpovědět, zda v zadaném grafu existuje párování, které obsahuje alespoň k hran. (Je jedno, zda se ptáme na párování o alespoň k hranách nebo o právě k , protože podmnožina párování je zase párování.)

⁽¹⁾ Jistě vás napadne spousta protipříkladů, jako třeba algoritmus se složitostí $\mathcal{O}(1.001^n)$, který nejspíš je použitelný, ačkoliv není polynomiální, a jiný se složitostí $\mathcal{O}(n^{100})$, u kterého je tomu naopak. Ukazuje se, že tyto případy jsou velmi řídké, takže u většiny problémů náš zjednodušený pohled funguje překvapivě dobře.

Abychom vyhověli definici, musíme určit, jak celý vstup problému zapsat jedním řetězcem bitů. Nabízí se očíslovat vrcholy grafu od 1 do n , hrany popsat maticí sousednosti a požadovanou velikost množiny k zapsat dvojkově. Musíme to ale udělat opatrně, abychom poznali, kde která část kódu začíná. Třeba takto:

$$\underbrace{\langle 11 \dots 10 \rangle}_t \underbrace{\langle n \text{ dvojkově} \rangle}_t \underbrace{\langle k \text{ dvojkově} \rangle}_t \underbrace{\langle \text{matice sousednosti} \rangle}_{n^2}$$

Počet jedniček na začátku kódu nám řekne, v kolika bitech je uložené n a k a zbytek kódu přečteme jako matici sousednosti.

Rozhodovací problém ovšem musí odpovědět na všechny řetězce bitů, nejen na ty ve správném tvaru. Dohodněme se tedy, že syntaxi vstupu budeme kontrolovat a na všechny chybně utvořené vstupy odpovíme nulou.

Jak párovací problém vyřešit? Jako správní matematici ho převedeme na nějaký, který už vyřešit umíme. To už jsme ostatně ukázali – umíme ho převést na toky v sítích. Pokaždé, když se ptáme na existenci párování velikosti alespoň k v nějakém bipartitním grafu, dovedeme sestavit určitou síť a zeptat se, zda v této síti existuje tok velikosti alespoň k . Překládáme tedy v polynomiálním čase vstup jednoho problému na vstup jiného problému, přičemž odpověď zůstane stejná.

Podobné převody mezi problémy můžeme definovat i obecněji:

Definice: Jsou-li A , B rozhodovací problémy, říkáme, že A lze převést na B (píšeme $A \rightarrow B$) právě tehdy, když existuje funkce $f : \{0, 1\}^* \rightarrow \{0, 1\}^*$ taková, že pro všechna $x \in \{0, 1\}^*$ platí $A(x) = B(f(x))$, a navíc lze funkci f spočítat v čase polynomiálním vzhledem k $|x|$. Funkci f říkáme *převod* nebo také *redukce*.

Pozorování: $A \rightarrow B$ také znamená, že problém B je alespoň tak těžký jako problém A (mnemotechnická pomůcka: obtížnost teče ve směru šipky). Tím myslíme, že kdykoliv umíme vyřešit B , je vyřešit A nanejvýš polynomiálně obtížnější. Speciálně platí:

Lemma: Pokud $A \rightarrow B$ a B lze řešit v polynomiálním čase, pak i A lze řešit v polynomiálním čase.

Důkaz: Nechť existuje algoritmus řešící problém B v čase $\mathcal{O}(b^k)$, kde b je délka vstupu tohoto problému a k konstanta. Mějme dále funkci f převádějící A na B v čase $\mathcal{O}(a^\ell)$ pro vstup délky a . Chceme-li nyní spočítat $A(x)$ pro nějaký vstup x délky a , spočítáme nejprve $f(x)$. To bude trvat $\mathcal{O}(a^\ell)$ a vyjde výstup délky taktéž $\mathcal{O}(a^\ell)$ – delší bychom v daném čase ani nestihli vypsát. Tento vstup pak předáme algoritmu pro problém B , který nad ním stráví čas $\mathcal{O}((a^\ell)^k) = \mathcal{O}(a^{k\ell})$. Celkový čas výpočtu proto činí $\mathcal{O}(a^\ell + a^{k\ell})$, což je polynom v délce původního vstupu. $\square$

Relace převoditelnosti tedy jistým způsobem porovnává problémy podle obtížnosti. Nabízí se představa, že se jedná o uspořádání na množině všech problémů. Je tomu doopravdy tak?

Pozorování: O relaci „ $\rightarrow$ “ platí:

- Je *reflexivní* ($A \rightarrow A$) – úlohu můžeme převést na tutéž identickým zobrazením.
- Je *tranzitivní* ($A \rightarrow B \wedge B \rightarrow C \Rightarrow A \rightarrow C$) – pokud funkce f převádí A na B a funkce g převádí B na C , pak funkce $g \circ f$ převádí A na C . Složení dvou polynomiálně vyčíslitelných funkcí je zase polynomiálně vyčíslitelná funkce, jak už jsme upozorovali v důkazu předchozího lemmatu.
- *Není antisymetrická* – například problémy „na vstupu je řetězec začínající nulou“ a „na vstupu je řetězec končící nulou“ lze mezi sebou převádět oběma směry.
- Existují *navzájem nepřevoditelné problémy* – třeba mezi problémy „na každý vstup odpověz 0“ a „na každý vstup odpověz 1“ nemůže existovat převod ani jedním směrem.

Relacím, které jsou reflexivní a tranzitivní, ale obecně nesplňují antisymetrii, se říká *kvaziuspořádání*. Převoditelnost je tedy částečné kvaziuspořádání na množině všech problémů.

Cvičení

1. Nahlédněte, že množina všech polynomů je nejmenší množina funkcí z $\mathbb{R}$ do $\mathbb{R}$, která obsahuje všechny konstantní funkce, identitu a je uzavřená na sčítání, násobení a skládání funkcí. Pokud tedy prohlásíme za efektivní právě polynomiální algoritmy, platí, že složením efektivních algoritmů (v mnoha možných smyslech) je zase efektivní algoritmus. To je velice příjemná vlastnost.
- 2.* Při kódování vstupu řetězcem bitů se často hodí umět zapsat číslo předem neznámé velikosti *instantním kódem*, tj. takovým, při jehož čtení poznáme, kdy skončil. Dvojkový zápis čísla x zabere $\lfloor \log_2 x \rfloor + 1$ bitů, ale není instantní. Kódování použité v problému párování je instantní a spotřebuje $2\lfloor \log_2 x \rfloor + \mathcal{O}(1)$ bitů. Navrhněte instantní kód, kterému stačí $\lfloor \log_2 x \rfloor + o(\log x)$ bitů. (Připomeňme, že $f \in o(g)$, pokud $\lim_{n \rightarrow \infty} f/g = 0$.)
- 3.* Převoditelnost je pouze kvaziuspořádání, ale můžeme z ní snadno vyrobit skutečné uspořádání: Definujeme relaci $A \sim B \equiv (A \rightarrow B) \wedge (B \rightarrow A)$. Dokažte, že je to ekvivalence, a relaci převoditelnosti zaveďte na třídách této ekvivalence. Taková převoditelnost už bude slabě antisymetrická. To je v matematice dost běžný trik, říká se mu *faktorizace* kvaziuspořádání. Vyzkoušejte si ho na relaci dělitelnosti na množině celých čísel.

19.2 Příklady převodů

Nyní se podíváme na příklady několika problémů, které se obecně považují za těžké. Uvidíme, že každý z nich je možné převést na všechny ostatní, takže z našeho „polynomiálního“ pohledu jsou stejně obtížné.

Problém SAT – splnitelnost (satisfiability) logických formulí v CNF

Mějme nějakou logickou formuli s proměnnými a logickými spojkami. Zajímá nás, je-li tato formule *splnitelná*, tedy zda lze za proměnné dosadit 0 a 1 tak, aby formule dala výsledek 1 (byla *splněna*).

Zaměříme se na formule ve speciálním tvaru, v takzvané *konjunktivní normální formě* (CNF):

- formule je složena z jednotlivých *klauzulí* oddělených spojkou $\wedge$,
- každá *klauzule* je složena z *literálů* oddělených $\vee$,
- každý *literál* je buďto proměnná, nebo její negace.

Vstup problému: Formule ψ v konjunktivní normální formě.

Výstup problému: Existuje-li dosazení 0 a 1 za proměnné tak, aby $\psi(\dots) = 1$.

Příklad: Formule $(x \vee y \vee z) \wedge (\neg x \vee y \vee z) \wedge (x \vee \neg y \vee z) \wedge (x \vee y \vee \neg z)$ je splnitelná, stačí nastavit například $x = y = z = 1$ (jaká jsou ostatní splňující ohodnocení?). Naproti tomu formule $(x \vee y) \wedge (x \vee \neg y) \wedge \neg x$ splnitelná není, což snadno ověříme třeba vyzkoušením všech čtyř možných ohodnocení.

Poznámka: Co kdybychom chtěli zjistit, zda je splnitelná nějaká formule, která není v CNF? V logice se dokazuje, že ke každé formuli lze najít ekvivalentní formuli v CNF, ale při tom se bohužel formule může až exponenciálně prodloužit. Později ukážeme, že pro každou formuli χ existuje nějaká formule χ' v CNF, která je splnitelná právě tehdy, když je χ splnitelná. Formule χ' přitom bude dlouhá $\mathcal{O}(|\chi|)$, ale budou v ní nějaké nové proměnné.

Problém 3-SAT – splnitelnost formulí s krátkými klauzulemi

Pro SAT zatím není známý žádný polynomiální algoritmus. Co kdybychom zkusili problém trochu zjednodušit a uvažovat pouze formule ve speciálním tvaru?

Definujeme problém 3-SAT, což je splnitelnost pro formule v CNF, jejichž každá klauzule obsahuje nejvýše tři literály. Ukážeme, že tento problém je stejně těžký jako původní SAT.

Převod 3-SAT $\rightarrow$ SAT: Jelikož 3-SAT je speciálním případem SATu, poslouží tu jako převodní funkce identita. (Implicitně předpokládáme, že oba problémy používají stejné kódování formulí do řetězců bitů a že na nekorektní vstupy odpovídají vždy záporně.)

Převod SAT $\rightarrow$ 3-SAT: Necht se ve formuli vyskytuje nějaká „dlouhá“ klauzule o $k > 3$ literálech. Můžeme ji zapsat ve tvaru $(\alpha \vee \beta)$, kde α obsahuje 2 literály a β $k - 2$ literálů. Pořídíme si novou proměnnou x a klauzuli nahradíme dvěma novými $(\alpha \vee x)$ a $(\beta \vee \neg x)$. První z nich obsahuje 3 literály, tedy je krátká. Druhá má $k - 1$ literálů, takže může být stále dlouhá, nicméně postup můžeme opakovat.

Takto postupně nahradíme všechny dlouhé klauzule krátkými, což bude trvat nejvýše polynomiálně dlouho, neboť klauzuli délky k rozebereme po $k - 3$ krocích.

Zbývá ukázat, že nová formule je splnitelná právě tehdy, byla-li splnitelná formule původní. K tomu stačí nahlédnout, že každý jednotlivý krok převodu splnitelnost zachovává.

Pokud původní formule byla splnitelná, uvažme nějaké splňující ohodnocení proměnných. Ukážeme, že vždy můžeme novou proměnnou x nastavit tak, aby vzniklo splňující ohodnocení nové formule.

Víme, že klauzule $(\alpha \vee \beta)$ byla splněna. Proto v daném ohodnocení:

- Buďto $\alpha = 1$. Pak položíme $x = 0$, takže $(\alpha \vee x)$ bude splněna díky α a $(\beta \vee \neg x)$ díky x .
- Anebo $\alpha = 0$, a tedy $\beta = 1$. Pak položíme $x = 1$, čímž bude $(\alpha \vee x)$ splněna díky x , zatímco $(\beta \vee \neg x)$ díky β .

Ostatní klauzule budou stále splněny.

V opačném směru: pokud dostaneme splňující ohodnocení nové formule, umíme z něj získat splňující ohodnocení formule původní. Ukážeme, že stačí zapomenout proměnnou x . Všechny klauzule, kterých se naše transformace netýká, jsou nadále splněné. Co klauzule $(\alpha \vee \beta)$?

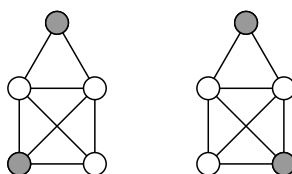
- Buďto $x = 0$, pak musí být $(\alpha \vee x)$ splněna díky α , takže $(\alpha \vee \beta)$ je také splněna díky α .
- Anebo $x = 1$, pak musí být $(\beta \vee \neg x)$ splněna díky β , takže i $(\alpha \vee \beta)$ je splněna.

Tím je převod hotov. SAT a 3-SAT jsou tedy stejně těžké.

Problém NzMna – nezávislá množina vrcholů v grafu

Definice: Množina vrcholů grafu je *nezávislá*, pokud žádné dva vrcholy ležící v této množině nejsou spojeny hranou. (Jinými slovy nezávislá množina indukuje podgraf bez hran.)

Na samotnou existenci nezávislé množiny se nemá smysl ptát – prázdná množina či libovolný jeden vrchol jsou vždy nezávislé. Zajímavé ale je, jestli graf obsahuje dostatečně velkou nezávislou množinu.



Obrázek 19.1: Největší nezávislé množiny

Vstup problému: Neorientovaný graf G a číslo $k \in \mathbb{N}$.

Výstup problému: Zda existuje nezávislá množina $A \subseteq V(G)$ velikosti alespoň k .

Převod 3-SAT $\rightarrow$ NzMna: Dostaneme formuli a máme vytvořit graf, v němž se bude nezávislá množina určené velikosti nacházet právě tehdy, je-li formule splnitelná. Myšlenka převodu bude jednoduchá: z každé klauzule budeme chtít vybrat jeden literál, jehož nastavením klauzuli splníme. Samozřejmě si musíme dát pozor, abychom v různých klauzulích nevybírali konfliktně, tj. jednou x a podruhé $\neg x$.

Jak to přesně zařídit: pro každou z k klauzulí zadané formule vytvoříme trojúhelník a jeho vrcholům přiřadíme literály klauzule. (Pokud by klauzule obsahovala méně literálů, prostě některé vrcholy trojúhelníku smažeme.) Navíc spojíme hranami všechny dvojice konfliktních literálů (x a $\neg x$) z různých trojúhelníků.

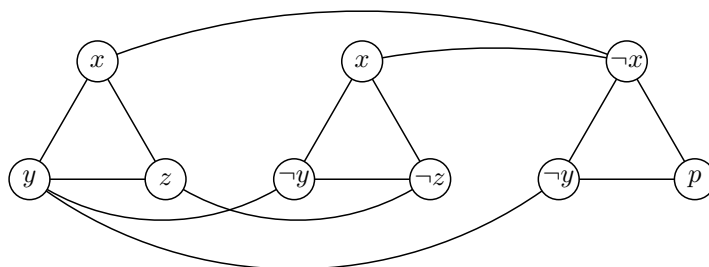
V tomto grafu se budeme ptát po nezávislé množině velikosti alespoň k . Jelikož z každého trojúhelníku můžeme do nezávislé množiny vybrat nejvýše jeden vrchol, jediná možnost, jak dosáhnout požadované velikosti, je vybrat z každého právě jeden vrchol. Ukážeme, že taková nezávislá množina existuje právě tehdy, je-li formule splnitelná.

Máme-li splňující ohodnocení formule, můžeme z každé klauzule vybrat jeden pravdivý literál. Do nezávislé množiny umístíme vrcholy odpovídající těmto literálům. Je jich právě k . Jelikož každé dva vybrané vrcholy leží v různých trojúhelnících a nikdy nemůže být pravdivý současně literál a jeho negace, množina je opravdu nezávislá.

A opačně: Kdykoliv dostaneme nezávislou množinu velikosti k , vybereme literály odpovídající vybraným vrcholům a příslušné proměnné nastavíme tak, abychom tyto literály splnili. Díky hranám mezi konfliktními literály se nikdy nestane, že bychom potřebovali proměnnou nastavit současně na 0 a na 1. Zbývající proměnné ohodnotíme libovolně. Jelikož jsme v každé klauzuli splnili alespoň jeden literál, jsou splněny všechny klauzule, a tedy i celá formule.

Převod je tedy korektní, zbývá rozmyslet si, že běží v polynomiálním čase: Počet vrcholů grafu odpovídá počtu literálů ve formuli, počet hran je maximálně kvadratický. Každý

vrchol i hranu přitom sestrojíme v polynomiálním čase, takže celý převod je také polynomiální.



Obrázek 19.2: Graf pro formuli $(x \vee y \vee z) \wedge (x \vee \neg y \vee \neg z) \wedge (\neg x \vee \neg y \vee p)$

Převod NzMna $\rightarrow$ SAT: Dostaneme graf a číslo k , chceme vytvořit formuli, která je splnitelná právě tehdy, pokud se v grafu nachází nezávislá množina o alespoň k vrcholech. Tuto formuli sestrojíme následovně.

Vrcholy grafu očíslovujeme od 1 do n a pořídíme si pro ně proměnné $v_1, \dots, v_n$, které budou indikovat, zda byl příslušný vrchol vybrán do nezávislé množiny (příslušné ohodnocení proměnných tedy bude odpovídat charakteristické funkci nezávislé množiny).

Aby množina byla opravdu nezávislá, pro každou hranu $ij \in E(G)$ přidáme klauzuli $(\neg v_i \vee \neg v_j)$.

Ještě potřebujeme zkontrolovat, že množina je dostatečně velká. To neumíme provést přímo, ale použijeme lest: vyrobíme matici proměnných $\mathbf{X}$ tvaru $k \times n$, která bude popisovat očíslování vrcholů nezávislé množiny čísla od 1 do k . Konkrétně $x_{i,j}$ bude říkat, že v pořadí i -tý prvek nezávislé množiny je vrchol j . K tomu potřebujeme zařídit:

- Aby v každém sloupci byla nejvýše jedna jednička. Na to si pořídíme klauzule $(x_{i,j} \Rightarrow \neg x_{i',j})$ pro $i' \neq i$. (Jsou to implikace, ale můžeme je zapsat i jako disjunkce, protože $a \Rightarrow b$ je totéž jako $\neg a \vee b$.)
- Aby v každém řádku ležela právě jedna jednička. Nejprve zajistíme nejvýše jednu klauzulemi $(x_{i,j} \Rightarrow \neg x_{i,j'})$ pro $j' \neq j$. Pak přidáme klauzule $(x_{i,1} \vee x_{i,2} \vee \dots \vee x_{i,n})$, které požadují alespoň jednu jedničku v řádku.
- Vztah mezi očíslováním a nezávislou množinou: přidáme klauzule $x_{i,j} \Rightarrow v_j$. (Všimněte si, že nezávislá množina může obsahovat i neočíslované prvky, ale to nám nevadí. Důležité je, aby jich měla k očíslovaných.)

Správnost převodu je zřejmá, ověříme ještě, že probíhá v polynomiálním čase. To plyne z toho, že vytvoříme polynomiálně mnoho klauzulí a každou z nich stihneme vypsát v lineárním čase.

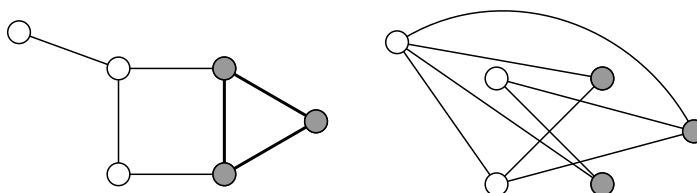
Dokázali jsme tedy, že testování existence nezávislé množiny je stejně těžké jako testování splnitelnosti formule. Pojdme se podívat na další problémy.

Problém Klika – úplný podgraf

Podobně jako nezávislou množinu můžeme v grafu hledat i *kliku* – úplný podgraf dané velikosti.

Vstup problému: Graf G a číslo $k \in \mathbb{N}$.

Výstup problému: Existuje-li úplný podgraf grafu G na alespoň k vrcholech.



Obrázek 19.3: Klika v grafu a nezávislá množina v jeho doplňku

Tento problém je ekvivalentní s hledáním nezávislé množiny. Pokud v grafu prohodíme hrany a nehrany, stane se z každé kliky nezávislá množina a naopak. Převodní funkce tedy znehybní hrany a ponechá číslo k .

Problém 3,3-SAT – splnitelnost s malým počtem výskytů

Než se pustíme do dalšího kombinatorického problému, předvedeme ještě jednu speciální variantu SATu, se kterou se nám bude pracovat příjemněji.

Již jsme ukázali, že SAT zůstane stejně těžký, omezíme-li se na formule s klauzulemi délky nejvýše 3. Teď budeme navíc požadovat, aby se každá proměnná vyskytovala v maximálně třech literálech. Tomuto problému se říká 3,3-SAT.

Převod 3-SAT $\rightarrow$ 3,3-SAT: Pokud se proměnná x vyskytuje v $k > 3$ literálech, nahradíme její výskyty novými proměnnými $x_1, \dots, x_k$ a přidáme klauzule, které zabezpečí, že tyto proměnné budou pokaždé ohodnoceny stejně: $(x_1 \Rightarrow x_2), (x_2 \Rightarrow x_3), (x_3 \Rightarrow x_4), \dots, (x_{k-1} \Rightarrow x_k), (x_k \Rightarrow x_1)$.

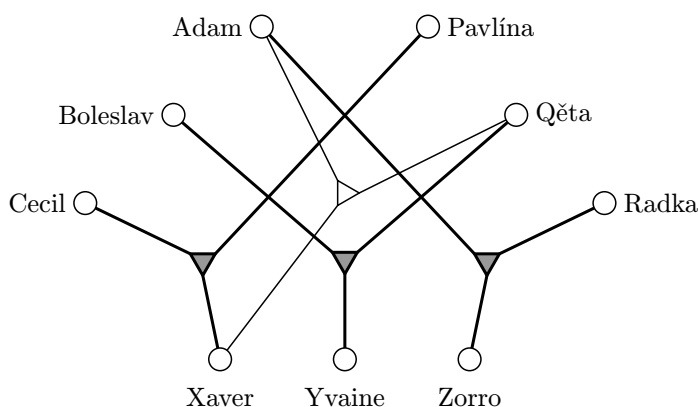
Zesílení: Můžeme dokonce zařídit, aby se každý literál vyskytoval nejvýše dvakrát (tedy že každá proměnná se vyskytuje alespoň jednou pozitivně a alespoň jednou negativně).

Pokud by se nějaká proměnná objevila ve třech stejných literálech, můžeme na ni také použít náš trik a nahradit ji třemi proměnnými. V nových klauzulích se pak bude vyskytovat jak pozitivně, tak negativně (opět připomínáme, že $a \Rightarrow b$ je jen zkratka za $\neg a \vee b$).

Problém 3D-párování

Vstup problému: Tři množiny, např. K (kluci), H (holky), Z (zvířátka) a množina $T \subseteq K \times H \times Z$ kompatibilních trojic (těch, kteří se spolu snesou).

Výstup problému: Zda existuje perfektní podmnožina trojic, tedy taková, v níž se každý prvek množin K , H a Z účastní právě jedné trojice.

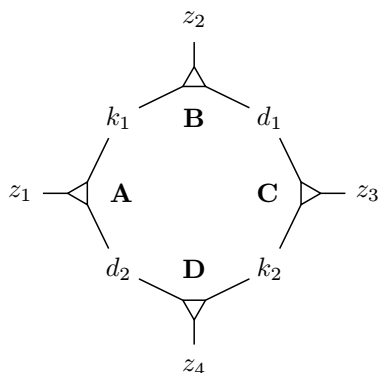


Obrázek 19.4: 3D-párování

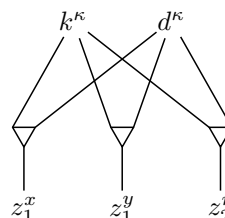
Převod 3,3-SAT $\rightarrow$ 3D-párování: Uvažujme trochu obecněji. Pokud chceme ukázat, že se na nějaký problém dá převést SAT, potřebujeme obvykle dvě věci: Jednak konstrukci, která bude simulovat proměnné, tedy něco, co nabývá dvou stavů 0/1. Poté potřebujeme cosi, co umí zařídit, aby každá klauzule byla splněna alespoň jednou proměnnou. Jak to provést u 3D-párování?

Uvažujme konfiguraci z obrázku 19.5. V ní se nacházejí 4 zvířátka (z_1 až z_4), 2 kluci (k_1 a k_2), 2 dívky (d_1 a d_2) a 4 trojice (A , B , C a D). Zatímco zvířátka se budou moci účastnit i jiných trojic, kluky a děvčata nikam jinam nezapojíme.

Všimneme si, že existují právě dvě možnosti, jak tuto konfiguraci spárovat. Abychom spárovali kluka k_1 , tak musíme vybrat buď trojici A nebo B . Pokud si vybereme A , k_1 i d_2 už jsou spárování, takže si nesmíme vybrat B ani D . Pak jediná možnost, jak spárovat d_1 a k_2 , je použít C . Naopak začneme-li trojicí B , vyloučíme A a C a použijeme D (situace je symetrická).



Obrázek 19.5: Konfigurace pro proměnnou



Obrázek 19.6: Konfigurace pro klauzuli

Vždy si tedy musíme vybrat dvě protější trojice v obrázku a druhé dvě nechat nevyužité. Tyto možnosti budeme používat k reprezentaci proměnných. Pro každou proměnnou si pořídíme jednu kopii obrázku. Volba $A + C$ bude odpovídat nule a nespáruje zvířátka z_2 a z_4 . Volba $B + D$ reprezentuje jedničku a nespáruje z_1 a z_3 . Přes tato nespárovaná zvířátka můžeme předávat informaci o hodnotě proměnné do klauzulí.

Zbývá vymyslet, jak reprezentovat klauzule. Mějme klauzuli tvaru řekněme $(x \vee y \vee \neg r)$. Potřebujeme zajistit, aby x bylo nastavené na 1 nebo y bylo nastavené na 1 nebo r na 0.

Pro takovou klauzuli přidáme konfiguraci z obrázku 19.6. Pořídíme si novou dvojici kluk a dívka, kteří budou figurovat ve třech trojicích se třemi různými zvířátky, což budou volná zvířátka z obrázků pro příslušné proměnné. Zvolíme je tak, aby se uvolnila při správném nastavení proměnné. Žádné zvířátko přitom nebude vystupovat ve více klauzulích, což můžeme splnit díky tomu, že každý literál se v 3,3-SATu vyskytuje nejvýše dvakrát a máme pro něj dvě volná zvířátka.

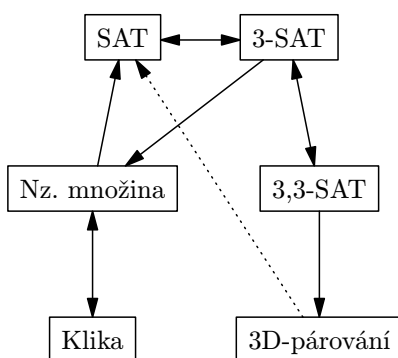
Ještě nám určitě zbude $4p - 3k$ zvířátek, kde p je počet proměnných a k počet klauzulí. Každá proměnná totiž dodá 4 volná zvířátka a každá klauzule použije 3 z nich. Přidáme proto ještě $4p - 3k$ párů lidí, kteří milují úplně všechna zvířátka; ti vytvoří zbývající trojice. Pokud je nějaká klauzule kratší než 3 literály, $3k$ se příslušně sníží.

Snadno ověříme, že celý převod pracuje v polynomiálním čase. Rozmysleme si ještě, že je korektní. Pokud formule byla splnitelná, z každého splňujícího ohodnocení můžeme vyrobit párování v naší konstrukci. Obrázek pro každou proměnnou spárujeme podle ohodnocení (buď $A + C$ nebo $B + D$). Pro každou klauzuli si vybereme trojici, která odpovídá některému z literálů, jimiž je klauzule splněna.

A opačně: Když nám někdo dá párování v naší konstrukci, dokážeme z něj vyrobit splňující ohodnocení dané formule. Podíváme se, v jakém stavu je proměnná, a to je všechno. Z toho, že jsou správně spárované klauzule, už okamžitě víme, že jsou všechny splněné.

Ukázali jsme tedy, že na 3D-párování lze převést 3,3-SAT, a tedy i obecný SAT. Převod v opačném směru ponecháme jako cvičení 2.

Jak je vidět ze schématu (obr. 19.7), všechny problémy z tohoto oddílu jsou navzájem převoditelné.



Obrázek 19.7: Problémy a převody mezi nimi

Cvičení

1. Domyslete detaily kódování vstupu pro libovolný z problémů z tohoto oddílu.
2. Převeďte 3D-párování na SAT. Jde to podobně, jako když jsme na SAT převáděli nezávislou množinu.
3. Co jsme ztratili omezením na rozhodovací problémy? Dokažte pro libovolný problém z tohoto oddílu, že pokud bychom ho dokázali v polynomiálním čase vyřešit, uměli bychom polynomiálně řešit i „zjišťovací“ verzi (najít konkrétní párování, splňující ohodnocení, kliku apod.).

Jak na to, ukážeme na párování: Chceme v grafu G najít párování velikosti k . Zvolíme libovolnou hranu e a otestujeme (zavoláním rozhodovací verze problému), zda i v grafu $G - e$ existuje párování velikosti k . Pokud ano, můžeme hranu e smazat a pokračovat dál. Pokud ne, znamená to, že hrana e leží ve všech párováních velikosti k , takže si ji zapamatujeme, smažeme z grafu včetně krajních vrcholů a všech incidentních hran, načež snížíme k o 1. Tak postupně získáme všech k hran párování.

4. *Vrcholové pokrytí* grafu je množina vrcholů, která obsahuje alespoň jeden vrchol z každé hrany. (Chceme na křižovatky rozmístit strážníky tak, aby každou ulici alespoň jeden hlídal.) Ukažte vzájemné převody mezi problémem nezávislé množiny a problémem „Existuje vrcholové pokrytí velikosti nejvýše k ?“.
5. Zesilte náš převod SATu na nezávislou množinu tak, aby vytvářel grafy, jejichž všechny vrcholy mají stupeň nejvýše 4.
6. Vyřešte SAT pro formule v disjunktivní normální formě (klauzule jsou spojené $\vee$, literály v klauzuli $\wedge$) v polynomiálním čase.

19.3 NP-úplné problémy

Všechny problémy, které jsme zatím zkoumali, měly jednu společnou vlastnost. Šlo v nich o to, zda existuje nějaký objekt. Například splňující ohodnocení formule nebo klika v grafu. Kdykoliv nám přitom někdo takový objekt ukáže, umíme snadno ověřit, že má požadovanou vlastnost. Ovšem najít ho už tak snadné není. Podobně se chovají i mnohé další „vyhledávací problémy“, zkusme je tedy popsat obecněji.

Definice: P je třída⁽²⁾ rozhodovacích problémů, které jsou řešitelné v polynomiálním čase. Jinak řečeno, problém L leží v P právě tehdy, když existuje nějaký algoritmus A a polynom f , přičemž pro každý vstup x algoritmus A doběhne v čase nejvýše $f(|x|)$ a vydá výsledek $A(x) = L(x)$.

Třída P tedy zachycuje naši představu o efektivně řešitelných problémech. Nyní definujeme třídu NP , která bude odpovídat naší představě vyhledávacích problémů.

Definice: NP je třída rozhodovacích problémů, v níž problém L leží právě tehdy, pokud existuje nějaký problém $K \in P$ a polynom g , přičemž pro každý vstup x je $L(x) = 1$ právě tehdy, pokud pro nějaký řetězec y délky nejvýše $g(|x|)$ platí $K(x, y) = 1$.⁽³⁾

Co to znamená? Algoritmus K řeší problém L , ale kromě vstupu x má k dispozici ještě polynomiálně dlouhou *nápovědu* y . Přitom má platit, že je-li $L(x) = 1$, musí existovat alespoň jedna nápověda, kterou algoritmus K schválí. Pokud ovšem $L(x) = 0$, nesmí ho přesvědčit žádná nápověda.

Jinými slovy y je jakýsi *certifikát*, který stvrzuje kladnou odpověď, a problém K má za úkol certifikáty kontrolovat. Pro kladnou odpověď musí existovat alespoň jeden schválený certifikát, pro zápornou musí být všechny certifikáty odmítnuty.

⁽²⁾ Formálně vzato je to množina, ale v teorii složitosti se pro množiny problémů vžil název *třída*.

⁽³⁾ Rozhodovací problémy mají na vstupu řetězec bitů. Tak jaképak x, y ? Máme samozřejmě na mysli nějaké binární kódování této dvojice.

To je podobné dokazování v matematice: k pravdivému tvrzení by měl existovat důkaz, u nepravdivého nás žádný „důkaz“ nepřesvědčí.

Příklad: Splnitelnost logických formulí je v NP. Stačí si totiž nechat napovědět, jak ohodnotit jednotlivé proměnné, a pak ověřit, je-li formule splněna. Náповěda je polynomiálně velká (dokonce lineárně), splnění zkontrolujeme také v lineárním čase. Podobně to lze dokázat i o ostatních rozhodovacích problémech, se kterými jsme v minulém oddílu potkali.

Pozorování: Třída P leží uvnitř NP. Pokud totiž problém umíme řešit v polynomiálním čase bez náповědy, tak to zvládneme v polynomiálním čase i s náповědou. Algoritmus K tedy bude ignorovat náповědy a odpověď spočítá přímo ze vstupu.

Nevíme ale, zda jsou třídy P a NP skutečně různé. Na to se teoretici informatiky snaží přijít už od 70. let minulého století a postupně se z toho stal snad vůbec nejslavnější otevřený problém informatiky.

Například pro žádný problém z předchozího oddílu nevíme, zda leží v P. Povede se nám ale dokázat, že tyto problémy jsou v jistém smyslu ty nejtěžší v NP.

Definice: Problém L nazveme NP-těžký, je-li na něj převoditelný každý problém z NP. Pokud navíc L leží v NP, budeme říkat, že L je NP-úplný.

Lemma: Pokud nějaký NP-těžký problém L leží v P, pak $P = NP$.

Důkaz: Již víme, že $P \subseteq NP$, takže stačí dokázat opačnou inkluzi. Vezměme libovolný problém $A \in NP$. Z NP-těžkosti problému L plyne $A \rightarrow L$. Už jsme ale dříve dokázali (lemma na straně 466), že pokud $L \in P$ a $A \rightarrow L$, pak také $A \in P$. $\square$

Existují ale vůbec nějaké NP-úplné problémy? Na první pohled zní nepravděpodobně, že by na nějaký problém z NP mohly jít převést všechny ostatní. Stephen Cook ale v roce 1971 dokázal následující překvapivou větu:

Věta (Cookova): SAT je NP-úplný.

Důkaz této věty je značně technický a alespoň v hrubých rysech ho předvedeme v příštím oddílu. Teď především ukážeme, že jakmile známe jeden NP-úplný problém, můžeme pomocí převoditelnosti dokazovat i NP-úplnost dalších.

Lemma: Mějme dva problémy $L, M \in NP$. Pokud L je NP-úplný a $L \rightarrow M$, pak M je také NP-úplný. (Intuitivně: Pokud L je nejtěžší v NP a $M \in NP$ je alespoň tak těžký jako L , pak M je také nejtěžší v NP.)

Důkaz: Jelikož M leží v NP, stačí o něm dokázat, že je NP-těžký, tedy že na něj lze převést libovolný problém z NP. Uvažme tedy nějaký problém $Q \in NP$. Jelikož L je NP-úplný,

musí platit $Q \rightarrow L$. Převoditelnost je ovšem tranzitivní, takže z $Q \rightarrow L$ a $L \rightarrow M$ plyne $Q \rightarrow M$. $\square$

Důsledek: Všechny problémy z minulého oddílu jsou NP-úplné.

Poznámka (o dvou možných světech): Jestli je $P = NP$, to nevíme a nejspíš ještě dlouho vědět nebudeme. Nechme se ale na chvíli unášet fantazií a zkusme si představit, jak by vypadaly světy, v nichž platí jedna nebo druhá možnost.

- $P = NP$ – to je na první pohled idylický svět, v němž jde každý vyhledávací problém vyřešit v polynomiálním čase, nejspíš tedy i prakticky efektivně. Má to i své stinné stránky: například jsme přišli o veškeré efektivní šifrování – rozmyslete si, že pokud umíme vypočítat nějakou funkci v polynomiálním čase, umíme efektivně spočítat i její inverzi.
- $P \neq NP$ – tehdy jsou P a NP-úplné dvě disjunktní třídy. SAT a ostatní NP-úplné problémy nejsou řešitelné v polynomiálním čase. Je ale stále možné, že aspoň na některé z nich existují prakticky použitelné algoritmy, třeba o složitosti $\Theta((1 + \varepsilon)^n)$ nebo $\Theta(n^{\log n/100})$. Také platí (tomu se říká Ladnerova věta [1]), že třída NP obsahuje i problémy, které svou obtížností leží někde mezi P a NP-úplnými.

Katalog NP-úplných problémů

Pokud se setkáme s problémem, který neumíme zařadit do P , hodí se vyzkoušet, zda je NP-úplný. K tomu se hodí mít alespoň základní zásobu „učebnicových“ NP-úplných problémů, abychom si mohli vybrat, z čeho převádět. U některých jsme už NP-úplnost dokázali, u ostatních alespoň naznačíme, jak na to.

- *Logické problémy:*
 - *SAT*: splnitelnost logických formulí v CNF
 - *3-SAT*: každá klauzule obsahuje max. 3 literály
 - *3,3-SAT*: navíc se každá proměnná vyskytuje nejvýše třikrát
 - *SAT pro obecné formule*: nejen v CNF (viz oddíl 19.4)
 - *Obvodový SAT*: splnitelnost booleovského obvodu (viz oddíl 19.4)
- *Grafové problémy:*
 - *Nezávislá množina*: existuje množina alespoň k vrcholů, mezi nimiž nevede žádná hrana?
 - *Klika*: existuje úplný podgraf na k vrcholech?
 - *Barvení grafu*: lze obarvit vrcholy k barvami (přidělit každému vrcholu číslo od 1 do k) tak, aby vrcholy stejné barvy nebyly nikdy spojeny hranou)? To je NP-úplné už pro $k = 3$, viz cvičení 2 a 3.

- *Hamiltonovská cesta*: existuje cesta obsahující všechny vrcholy? (cvičení 5)
- *Hamiltonovská kružnice*: existuje kružnice obsahující všechny vrcholy? (cvičení 4)
- *3D-párování*: máme tři množiny se zadanými trojicemi; zjistěte, zda existuje taková množina disjunktních trojic, ve které jsou všechny prvky právě jednou? (Striktně vzato, není to grafový problém, ale hypergrafový – hrany nejsou páry, ale trojice.)
- *Číselné problémy*:
 - *Součet podmnožiny*: má daná množina přirozených čísel podmnožinu s daným součtem? (cvičení 8)
 - *Batoh*: jsou dány předměty s váhami a cenami a kapacita batohu, chceme najít co nejdražší podmnožinu předmětů, jejíž váha nepřesáhne kapacitu batohu. Aby se jednalo o rozhodovací problém, ptáme se, zda existuje podmnožina s cenou větší nebo rovnou zadanému číslu. (cvičení 9)
 - *Dva loupežníci*: lze rozdělit danou množinu čísel na dvě podmnožiny se stejným součtem? (cvičení 7)
 - $\mathbf{Ax} = \mathbf{1}$ (*soustava nula-jedničkových lineárních rovnic*): je dána matice $\mathbf{A} \in \{0, 1\}^{m \times n}$. Existuje vektor $\mathbf{x} \in \{0, 1\}^n$ takový, že $\mathbf{Ax}$ je rovno vektoru samých jedniček? (cvičení 1)

Další třídy problémů*

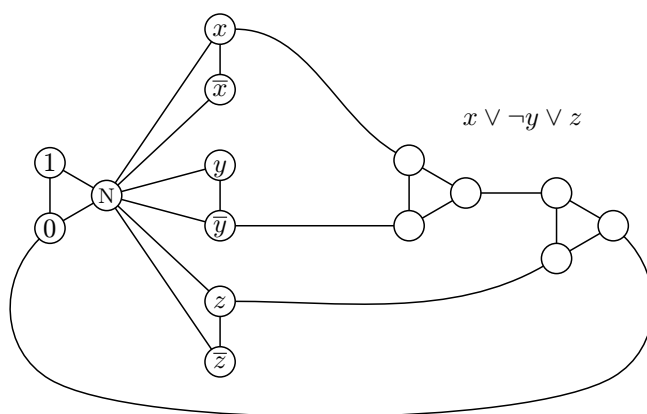
Třída P je uzavřená na negaci: kdykoliv nějaký problém L leží v P , pak jeho negace také leží v P . Pro třídu NP to ale tak samozřejmě není: ze schopnosti certifikovat kladnou odpověď vůbec neplyne schopnost certifikovat zápornou. Například u SATu kladnou odpověď certifikujeme tím, že ukážeme splňující ohodnocení. Ale jak ukázat, že formule není splnitelná?

To vede k definici třídy $co-NP$. Tam patří všechny problémy, jejichž negace leží v NP , tedy je možné certifikovat *negativní* odpovědi. Opět platí $P \subseteq co-NP$ a neví se, zda inkluze je ostrá. Také se neví, zda NP a $co-NP$ jsou různé třídy. I v třídě $co-NP$ funguje převoditelnost a existují úplné problémy: jeden z nich prozkoumáme ve cvičení 14.

Také můžeme obtížnost problémů měřit pomocí prostorové složitosti. Do třídy $PSPACE$ patří ty problémy, které jsou řešitelné algoritmem pracujícím v polynomiálním prostoru. Třída $PSPACE$ evidentně obsahuje P , ovšem také NP i $co-NP$ (cvičení 15). Ačkoliv $PSPACE$ vypadá mnohem obsáhleji, překvapivě se ani u této třídy neví, zda je různá od P . Převoditelnost i úplnost stále fungují. Příkladem $PSPACE$ -úplného problému je QSAT – varianta SATu pro formule s kvantifikátory $\forall$ a $\exists$ (proměnné jsou nicméně stále booleovské). Úplnost zde nicméně nebudeme dokazovat.

Cvičení

1. Dokažte NP-úplnost problému $\mathbf{Ax} = \mathbf{1}$. [N94]
- 2.* Dokažte NP-úplnost problému *barvení grafu třemi barvami* převodem z 3-SATu. Inspirujte se obrázkem 19.8.

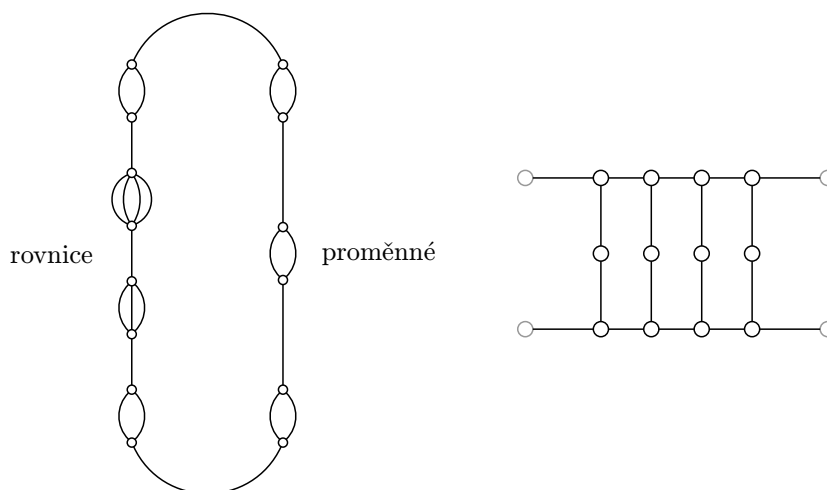


Obrázek 19.8: K důkazu NP-úplnosti barvení grafu

3. Ukažte, že *barvení grafu jednou nebo dvěma barvami* leží v P.
- 4.* Dokažte NP-úplnost problému *hamiltonovské kružnice* podle následujícího návodu. Nejprve budeme převádět problém $\mathbf{Ax} = \mathbf{1}$ na speciální variantu hamiltonovské kružnice. V té budou povoleny paralelní hrany a bude možné dvojice hran *párovat* – určit, že hledaná kružnice projde právě jednou hranou z dvojice.

Pro danou soustavu rovnic sestojíme graf podle obrázku 19.9 vlevo. Pro každou proměnnou pořídíme dvě paralelní hrany, které budou odpovídat jejím možným hodnotám. Pro každou rovnici pořídíme paralelní hrany odpovídající jednotlivým výskytům proměnných (s nenulovým koeficientem). Nakonec spárujeme každý výskyt proměnné v rovnici s nulovým stavem příslušné proměnné.

Dokažte, že v tomto grafu existuje párovací hamiltonovská kružnice právě tehdy, má-li soustava rovnic řešení. Vymyslete, jak se zbavit paralelních hran a požadavků na párování. K tomu se může hodit rozmyslet si, jak může hamiltonovská kružnice procházet podgrafem z obrázku 19.9 vpravo.



Obrázek 19.9: K důkazu NP-úplnosti hamiltonovské kružnice

5. Upravte konstrukci z předchozího cvičení na důkaz NP-úplnosti problému *hamiltonovské cesty*. Uvažujte i verzi problému, v níž máme pevně určený jeden nebo oba krajní vrcholy cesty.
6. Dokažte, že uvedené problémy týkající se hamiltonovských cest a kružnic zůstávají NP-úplné i pokud všechny vrcholy mají stupeň nejvýše 3.
7. Převeďte součet podmnožiny na dva loupežníky a opačně.
8. Dokažte NP-úplnost problému *součtu podmnožiny*. [N34]
9. Dokažte NP-úplnost problému batohu. [N54]
10. Pokud bychom definovali P-úplnost analogicky k NP-úplnosti, které problémy z P by byly P-úplné?
11. Převeďte libovolný problém z katalogu na SAT, aniž byste použili Cookovu větu.
12. *Kvadratické rovnice*: Převeďte SAT na řešitelnost soustavy kvadratických rovnic více proměnných, tedy rovnic tvaru

$$\sum_i \alpha_i x_i^2 + \sum_{i,j} \beta_{ij} x_i x_j + \sum_i \gamma_i x_i + \delta = 0,$$

kde $x_1, \dots, x_n$ jsou reálné neznámé a řecká písmena značí celočíselné konstanty. (Všimněte si, že vůbec není jasné, zda tento problém leží v NP.)

13. *Množinové pokrytí*: Na vstupu dostaneme množiny $A_1, \dots, A_k \subseteq X$ a číslo t . Úkolem je zjistit, zda lze vybrat t -tici množin tak, aby jejich sjednocení bylo celé X . Dokažte, že tento problém je NP-úplný.
- 14.* *Tautologie* říkáme formuli, která je pravdivá pro každé ohodnocení proměnných. Dokažte, že problém, zda zadaná formule je tautologie, je co-NP-úplný. [N53]
- 15.* Ukažte, že libovolný problém z NP nebo co-NP leží v PSPACE. [N31]
- 16.* Dokažte, že QSAT leží v PSPACE.
- 17.* Necht pro problém L existuje pravděpodobnostní algoritmus, který běží v polynomiálním čase (v nejhorším případě) a vydá správnou odpověď s pravděpodobností větší než $1/2$. Dokažte, že pak L leží v PSPACE. [N25]

19.4* Důkaz Cookovy věty

Zbývá dokázat Cookovu větu. Potřebujeme ukázat, že SAT je NP-úplný, a to přímo z definice NP-úplnosti. Nejprve se nám to povede pro jiný problém, pro takzvaný *obvodový SAT*. V něm máme na vstupu booleovský obvod (hradlovou síť) s jedním výstupem a ptáme se, zda můžeme přivést na vstupy obvodu takové hodnoty, aby vydal výsledek 1. To je obecnější než SAT pro formule (dokonce i neomezíme-li formule na CNF), protože každou formuli můžeme přeložit na lineárně velký obvod (cvičení 15.1.11).

Nejprve tedy dokážeme NP-úplnost obvodového SATu a pak ho převedeme na obyčejný SAT v CNF. Tím bude důkaz Cookovy věty hotov. Začneme lemmatem, v němž bude koncentrováno vše technické.

Budeme se snažit ukázat, že pro každý problém v P existuje polynomiálně velká hradlová síť, která ho řeší. Jenom si musíme dát pozor na to, že pro různé velikosti vstupu potřebujeme různé hradlové sítě, které navíc musíme umět efektivně generovat.

Lemma: Necht L je problém ležící v P. Potom existuje polynom p a algoritmus, který pro každé n sestrojí v čase $p(n)$ hradlovou síť B_n s n vstupy a jedním výstupem, která řeší L . Tedy pro všechny řetězce $x \in \{0, 1\}^n$ musí platit $B_n(x) = L(x)$.

Náznak důkazu: Vyjdeme z intuice o tom, že počítače jsou jakési složité booleovské obvody, jejichž stav se mění v čase. (Formálněji bychom konstruovali booleovský obvod simulující výpočetní model RAM.)

Uvažme tedy nějaký problém $L \in P$ a polynomiální algoritmus, který ho řeší. Pro vstup velikosti n algoritmus doběhne v čase T polynomiálním v n a spotřebuje $\mathcal{O}(T)$ buněk paměti. Stačí nám tedy „počítač s pamětí velkou $\mathcal{O}(T)$ “, což je nějaký booleovský obvod velikosti polynomiální v T , a tedy i v n . Vývoj v čase ošetříme tak, že sestrojíme T kopií tohoto obvodu, každá z nichž bude odpovídat jednomu kroku výpočtu a bude propojena s „minulou“ a „budoucí“ kopií. Tím sestrojíme booleovský obvod, který bude řešit problém L pro vstupy velikosti n a bude polynomiálně velký vzhledem k n . $\square$

Úprava definice NP: Pro důkaz následující věty si dovolíme drobnou úpravu v definici třídy NP. Budeme chtít, aby nápověda měla pevnou velikost, závislou pouze na velikosti vstupu (tedy: $|y| = g(|x|)$ namísto $|y| \leq g(|x|)$). Proč je taková úprava bez újmy na obecnosti? Stačí původní nápovědu doplnit na požadovanou délku nějakými „mezerami“, které budeme při ověřování nápovědy ignorovat. Podobně můžeme zaokrouhlit koeficienty polynomu g na celá čísla, aby ho bylo možné vyhodnotit v konstantním čase.

Věta (téměř Cookova): Obvodový SAT je NP-úplný.

Důkaz: Obvodový SAT evidentně leží v NP – stačí si nechat poradit vstup, síť topologicky seřadit a v tomto pořadí počítat hodnoty hradel.

Mějme nyní nějaký problém L z NP, o němž chceme dokázat, že se dá převést na obvodový SAT. Když nám někdo předloží nějaký vstup x délky n , spočítáme velikost nápovědy $g(n)$. Víme, že algoritmus K , který kontroluje, zda nápověda je správně, leží v P. Využijeme předchozí lemma, abychom získali obvod, který pro konkrétní velikost vstupu n počítá to, co kontrolní algoritmus K . Vstupem tohoto obvodu bude x (vstup problému L) a nápověda y . Na výstupu se dozvíme, zda je nápověda správná. Velikost tohoto obvodu bude činit $p(n + g(n))$, což je také polynom.

V tomto obvodu zafixujeme vstup x (na místa vstupu dosadíme konkrétní hodnoty z x). Tím získáme obvod, jehož vstup je jen y , a chceme zjistit, zda za y můžeme dosadit nějaké hodnoty tak, aby na výstupu byla 1. Jinými slovy, ptáme se, zda je tento obvod splnitelný.

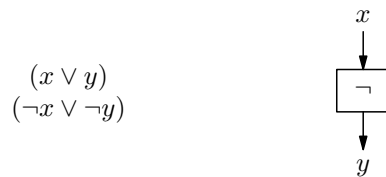
Ukázali jsme tedy, že pro libovolný problém z NP dokážeme sestrojit funkci, která pro každý vstup x v polynomiálním čase vytvoří obvod, jenž je splnitelný právě tehdy, když odpověď tohoto problému na vstup x má být kladná. To je přesně převod z daného problému na obvodový SAT. $\square$

Lemma: Obvodový SAT se dá převést na 3-SAT.

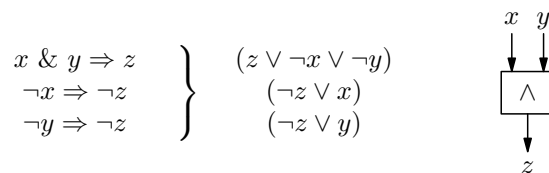
Důkaz: Budeme postupně budovat formuli v konjunktivní normální formě. Každý booleovský obvod se dá v polynomiálním čase převést na ekvivalentní obvod, ve kterém se vyskytují jen hradla AND a NOT (cvičení 15.1.3), takže stačí najít klauzule odpovídající těmto hradlům. Pro každé hradlo v obvodu zavedeme novou proměnnou popisující

jeho výstup. Přidáme klauzule, které nám kontrolují, že toto hradlo máme ohodnocené konzistentně.

Převod hradla NOT: Na vstupu hradla budeme mít nějakou proměnnou x (která přišla buďto přímo ze vstupu celého obvodu, nebo je to výstup nějakého jiného hradla) a na výstupu proměnnou y . Přidáme klauzule, které nám zaručí, že jedna proměnná bude negací té druhé:



Převod hradla AND: Hradlo má vstupy x, y a výstup z . Potřebujeme přidat klauzule, které nám popisují, jak se má hradlo AND chovat. Tyto vztahy přepíšeme do konjunktivní normální formy:



Tím v polynomiálním čase vytvoříme formuli, která je splnitelná právě tehdy, je-li splnitelný zadaný obvod. Ve splňujícím ohodnocení formule bude obsaženo jak splňující ohodnocení obvodu, tak výstupy všech hradel obvodu. $\square$

Poznámka: Tím jsme také odpověděli na otázku, kterou jsme si kladli při zavádění SATu: tedy zda omezením na CNF o něco přijdeme. Teď už víme, že nepřijdeme – libovolná booleovská formule se dá přímočaře převést na obvod a ten zase na formuli v CNF. Zavádíme sice nové proměnné, ale nová formule je splnitelná právě tehdy, kdy ta původní.

Cvičení

- 1.* Dokažte lemma o vztahu mezi problémy z P a hradlovými sítěmi pomocí výpočetního modelu RAM.

19.5 Co si počít s těžkým problémem

NP-úplné problémy jsou obtížné, nicméně v životě velmi běžné. Přesněji řečeno spíš než s rozhodovacím problémem se potkáme s problémem *optimalizačním*, ve kterém jde o nalezení *nejlepšího* objektu s danou vlastností. To může být třeba největší nezávislá množina v grafu nebo obarvení grafu nejmenším možným počtem barev. Kdybychom uměli efektivně řešit optimalizační problém, umíme samozřejmě řešit i příslušný rozhodovací, takže pokud $P \neq NP$, jsou i optimalizační problémy těžké.

Ale co naplat, svět nám takové úlohy předkládá a my je potřebujeme vyřešit. Naštěstí situace není zase tak beznadějná. Nabízejí se tyto možnosti, co si počít:

1. *Spokojit se s málem.* Nejsou vstupy, pro které problém potřebujeme řešit, dostatečně malé, abychom si mohli dovolit použít algoritmus s exponenciální složitostí? Zvlášť když takový algoritmus vylepšíme prořezáváním neperspektivních větví výpočtu a třeba ho i paralelizujeme.
2. *Vyřešit speciální případ.* Nemají naše vstupy nějaký speciální tvar, kterého bychom mohli využít? Grafové problémy jsou často v P třeba pro stromy nebo i obecněji pro bipartitní grafy. U číselných problémů zase někdy pomůže, jsou-li čísla na vstupu dostatečně malá.
3. *Řešení aproximovat.* Opravdu potřebujeme optimální řešení? Nestačilo by nám o kousíček horší? Často existuje polynomiální algoritmus, který nalezne nejhůře c -krát horší řešení, než je optimum, přičemž c je konstanta.
4. *Použít heuristiku.* Neumíme-li nic lepšího, můžeme sáhnout po některé z mnoha heuristických technik, které sice nic nezaručují, ale obvykle nějaké uspokojivé řešení najdou. Může pomoci třeba hladový algoritmus nebo evoluční algoritmy. Často platí, že čím déle heuristiku necháme běžet, tím lepší řešení najde.
5. *Kombinace přístupů.* Mnohdy lze předchozí přístupy kombinovat: například použít aproximační algoritmus a poté jeho výsledek ještě heuristicky vylepšovat. Tak získáme řešení, které od optima zaručeně není moc daleko, a pokud budeme mít štěstí, bude se od něj lišit jen velmi málo.

Nyní si některé z těchto technik předvedeme na konkrétních příkladech.

Největší nezávislá množina ve stromu

Ukážeme, že hledání největší nezávislé množiny je snadné, pokud graf je strom, nebo dokonce les.

Lemma: Buď T les a ℓ jeho libovolný list. Pak alespoň jedna z největších nezávislých množin obsahuje ℓ .

Důkaz: Mějme největší nezávislou množinu M , která list ℓ neobsahuje. Podívejme se na souseda p listu ℓ . Leží p v M ? Pokud ne, mohli bychom do M přidat list ℓ a dostali bychom větší nezávislou množinu. V opačném případě odebereme z M souseda p a nahradíme ho listem ℓ , čímž dostaneme stejně velkou nezávislou množinu obsahující ℓ . $\square$

Algoritmus bude přímočaře používat toto lemma. Dostane na vstupu les a najde v něm libovolný list. Tento list umístí do nezávislé množiny a jeho souseda z lesa smaže, protože se nemůže v nezávislé množině vyskytovat. Toto budeme opakovat, dokud nějaké listy zbyvají. Zbylé izolované vrcholy také přidáme do nezávislé množiny.

Tento algoritmus jistě pracuje v polynomiálním čase. Šikovnou implementací můžeme složitost snížit až na lineární, například tak, že budeme udržovat seznam listů. My zde ukážeme jinou lineární implementaci založenou na prohledávání do hloubky. Bude pracovat s polem značek M , v němž na počátku bude všude *false* a postupně obdrží *true* všechny prvky hledané nezávislé množiny.

Algoritmus NZMNAVESTROMU

Vstup: Strom T s kořenem v , pole značek M

1. $M[v] \leftarrow \text{true}$.
2. Pokud je v list, skončíme.
3. Pro všechny syny w vrcholu v :
4. Zavoláme se rekurzivně na podstrom s kořenem w .
5. Pokud $M[w] = \text{true}$, položíme $M[v] \leftarrow \text{false}$.

Výstup: Pole M indikující nezávislou množinu

Můžeme si představit, že při návratu z rekurze vrcholy odstraňujeme. Kdykoliv se chystáme vrátit z vrcholu v a některý z jeho synů leží v nezávislé množině, představíme si, že jsme v smazali. Naopak neleží-li žádný syn v nezávislé množině, v se chová jako list, takže ho chceme do nezávislé množiny přidat a ze stromu odtrhnout.

Barvení intervalového grafu

Mějme n přednášek s určenými časy začátku a konce. Chceme je rozvrhnout do co nejmenšího počtu poslucháren tak, aby nikdy neprobíhaly dvě přednášky naráz v jedné místnosti.

Chceme tedy obarvit co nejmenším počtem barev graf, jehož vrcholy jsou časové intervaly a dvojice intervalů je spojena hranou, pokud má neprázdný průnik. Takovým grafům se říká *intervalové* a pro jejich barvení existuje pěkný polynomiální algoritmus.

Podobně jako jsme geometrické problémy řešili zametáním roviny, zde budeme „zametati přímkou bodem“, tedy procházet ji zleva doprava, a všimnout si událostí, což budou začát-

ky a konce intervalů. Pro jednoduchost předpokládejme, že všechny souřadnice začátků a konců jsou navzájem různé.

Kdykoliv interval začne, přidělíme mu barvu. Až skončí, o barvě si poznamenejme, že je momentálně volná. Dalším intervalům budeme přednostně přidělovat volné barvy. Řečeno v pseudokódu:

Algoritmus BARVENÍINTERVALŮ

Vstup: Intervaly $[x_1, y_1], \dots, [x_n, y_n]$

1. $b \leftarrow 0$ $\triangleleft$ počet zatím použitých barev
2. $B \leftarrow \emptyset$ $\triangleleft$ které barvy jsou momentálně volné
3. Setřídíme množinu všech x_i a y_i .
4. Procházíme všechna x_i a y_i ve vzestupném pořadí:
5. Narazíme-li na x_i :
6. Je-li $B \neq \emptyset$, odebereme jednu barvu z B a uložíme ji do c_i .
7. Jinak $b \leftarrow b + 1$ a $c_i \leftarrow b$.
8. Narazíme-li na y_i :
9. Vrátíme barvu c_i do B .

Výstup: Obarvení $c_1, \dots, c_n$

Tento algoritmus má časovou složitost $\mathcal{O}(n \log n)$ kvůli třídění souřadnic. Samotné obarvování je lineární.

Ještě ovšem potřebujeme dokázat, že jsme použili minimální možný počet barev. Uvažujme okamžik, kdy proměnná b naposledy vzrostla. Tehdy začal interval a množina B byla prázdná, což znamená, že jsme $b - 1$ předchozích barev museli přidělit intervalům, jež začaly a dosud neskončily. Existuje tedy b různých intervalů, které mají společný bod (v grafu tvoří kliku), takže každé obarvení potřebuje alespoň b barev.

Problém batohu s malými čísly

Připomeňme si *problém batohu*. Jeho optimalizační verze vypadá takto: Je dána množina n předmětů s hmotnostmi $h_1, \dots, h_n \in \mathbb{N}$ a cenami $c_1, \dots, c_n \in \mathbb{N}$ a nosnost batohu H . Hledáme podmnožinu předmětů $P \subseteq \{1, \dots, n\}$, která se vejde do batohu (tedy $h(P) \leq H$, kde $h(P) := \sum_{i \in P} h_i$) a její cena $c(P) := \sum_{i \in P} c_i$ je největší možná.

Ukážeme algoritmus, jehož časová složitost bude polynomiální v počtu předmětů n a součtu všech cen $C = \sum_i c_i$.

Použijeme dynamické programování. Představme si problém omezený na prvních k předmětů. Označme $A_k(c)$ (kde $0 \leq c \leq C$) minimum z hmotností těch podmnožin, jejichž cena je právě c ; pokud žádná taková podmnožina neexistuje, položíme $A_k(c) = \infty$.

Tato A_k spočteme indukcí podle k : Pro $k = 0$ je určitě $A_0(0) = 0$ a $A_0(1) = A_0(2) = \dots = A_0(C) = \infty$. Pokud již známe A_{k-1} , spočítáme A_k následovně: $A_k(c)$ odpovídá nějaké podmnožině předmětů z $1, \dots, k$. V této podmnožině jsme buďto k -tý předmět nepoužili, a pak je $A_k(c) = A_{k-1}(c)$, nebo použili, a tehdy bude $A_k(c) = A_{k-1}(c - c_k) + h_k$ (to samozřejmě jen pokud $c \geq c_k$). Z těchto dvou možností si vybereme tu, která dává množinu s menší hmotností:

$$A_k(c) = \min(A_{k-1}(c), A_{k-1}(c - c_k) + h_k).$$

Přechod od A_{k-1} k A_k tedy trvá $\mathcal{O}(C)$, od A_1 až k A_n se dopočítáme v čase $\mathcal{O}(Cn)$.

Jakmile získáme A_n , známe pro každou cenu příslušnou nejlehčí podmnožinu. Maximální cena množiny, která se vejde do batohu, je tedy největší c^* , pro něž je $A_n(c^*) \leq H$. Jeho nalezení nás stojí čas $\mathcal{O}(C)$.

Zbývá zjistit, které předměty do nalezené množiny patří. Upravíme algoritmus, aby si pro každé $A_k(c)$ pamatoval ještě $B_k(c)$, což bude index posledního předmětu, který jsme do příslušné množiny přidali. Pro nalezené c^* tedy bude $i = B_n(c^*)$ poslední předmět v nalezené množině, $i' = B_{i-1}(c^* - c_i)$ ten předposlední a tak dále. Takto v čase $\mathcal{O}(n)$ rekonstruujeme celou množinu od posledního prvku k prvnímu.

Máme tedy algoritmus, který vyřeší problém batohu v čase $\mathcal{O}(nC)$. Tato funkce ovšem není polynomem ve velikosti vstupu: reprezentujeme-li vstup binárně, C může být až exponenciálně velké vzhledem k délce jeho zápisu. To je pěkný příklad tzv. *pseudopolynomiálního* algoritmu, tedy algoritmu, jehož složitost je polynomem v počtu čísel na vstupu a jejich velikosti. Pro některé NP-úplné problémy takové algoritmy existují, pro jiné (např. pro nezávislou množinu) by z jejich existence plynulo $P = NP$.

Problém batohu bez cen

Někdy se uvažuje též zjednodušená verze problému batohu, v níž nerozlišujeme mezi hmotnostmi a cenami. Chceme tedy do batohu naskládat nejtěžší podmnožinu, která se tam ještě vejde. Tento problém zvládneme pro malá čísla vyřešit i jiným algoritmem, opět založeným na dynamickém programování.

Indukcí podle k vytváříme množiny Z_k obsahující všechny hmotnosti menší než H , kterých nabývá nějaká podmnožina prvních k prvků. Jistě je $Z_0 = \{0\}$. Podobnou úvahou jako v předchozím algoritmu dostaneme, že každou další Z_k můžeme zapsat jako sjednocení Z_{k-1} s kopií Z_{k-1} posunutou o h_k , ignorující hodnoty větší než H . Nakonec ze Z_n vyčteme výsledek.

Všechny množiny přitom mají nejvýše $H + 1$ prvků, takže pokud si je budeme udržovat jako seřazené seznamy, spočítáme sjednocení sléváním v čase $\mathcal{O}(H)$ a celý algoritmus doběhne v čase $\mathcal{O}(nH)$.

Cvičení

1. Popište polynomiální algoritmus pro hledání nejmenšího *vrcholového pokrytí stromu*. (To je množina vrcholů, která obsahuje alespoň jeden vrchol z každé hrany.)
- 2.* Nalezněte polynomiální algoritmus pro hledání nejmenšího *vrcholového pokrytí bipartitního grafu*. [N74]
3. Vážená verze nezávislé množiny: Vrcholy stromu mají celočíselné *váhy*, hledáme nezávislou množinu s maximálním součtem vah. [N101]
4. Ukažte, jak v polynomiálním čase najít největší *nezávislou množinu v intervalovém grafu*.
- 5.* Vyřešte v polynomiálním čase 2-SAT, tedy splnitelnost formulí zadaných v CNF, jejichž klauzule obsahují nejvýše 2 literály. [N99]
6. Problém E3, E3-SAT je dalším zesílením 3,3-SATu. Chceme zjistit splnitelnost formule v CNF, jejíž každá klauzule obsahuje právě tři různé proměnné a každá proměnná se nachází v právě třech klauzulích. Ukažte, že tento problém lze řešit efektivně z toho prostého důvodu, že každá taková formule je splnitelná. [N49]
7. *Hladoví loupežníci*: Pokusíme se řešit problém dvou loupežníků hladovým algoritmem. Probíráme předměty od nejdražšího k nejlevnějšímu a každý dáme tomu loupežníkovi, který má zrovna méně. Je nalezené řešení optimální?
8. *Problém tří loupežníků*: Je dána množina předmětů s cenami, chceme ji rozdělit na 3 části o stejné ceně. Navrhněte pseudopolynomiální algoritmus.
9. Upravte algoritmus z části „Problém batohu bez cen“, aby zvládl i verzi s cenami v čase $\mathcal{O}(nH)$.

19.6 Aproximační algoritmy

Neumíme-li najít přesné řešení problému, ještě není vše ztraceno: můžeme ho *aproximovat*. Co to znamená?

Optimalizační problémy obvykle vypadají tak, že mají nějakou množinu *přípustných řešení*, každé z nich ohodnoceno nějakou *cenou* $c(x)$. Mezi nimi hledáme *optimální řešení* s minimální cenou c^* . Zde si vystačíme s jeho α -*aproximací*, čili s přípustným řešením s cenou $c' \leq \alpha c^*$ pro nějakou konstantu $\alpha > 1$. To je totéž jako říci, že relativní chyba $(c' - c^*)/c^*$ nepřekročí $\alpha - 1$.

Analogicky bychom mohli studovat maximalizační problémy a chtít alespoň α -násobek optima pro $0 < \alpha < 1$.

Aproximace problému obchodního cestujícího

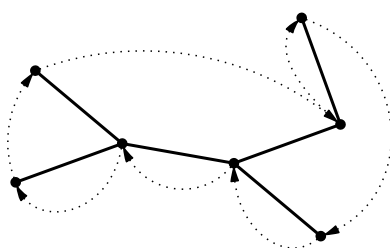
V problému obchodního cestujícího je zadán neorientovaný graf G , jehož hrany jsou ohodnoceny délkami $\ell(e) \geq 0$. Chceme nalézt nejkratší z hamiltonovských kružnic, tedy těch, které navštíví všechny vrcholy. (Obchodní cestující chce navštívit všechna města na mapě a najezdit co nejméně.)

Není překvapivé, že tento problém je těžký – už sama existence hamiltonovské kružnice je NP-úplná. Nyní ukážeme, že pokud je graf úplný a platí v něm trojúhelníková nerovnost (tj. $\ell(x, z) \leq \ell(x, y) + \ell(y, z)$ pro všechny trojice vrcholů x, y, z), můžeme problém obchodního cestujícího 2-aproximovat. To znamená najít v polynomiálním čase kružnici, která je přinejhorším dvakrát delší než ta optimální.

Grafy s trojúhelníkovou nerovností přitom nejsou nijak neobvyklé – odpovídají konečným metrickým prostorům.

Algoritmus bude snadný: Najdeme nejmenší kostru a obchodnímu cestujícímu poradíme, ať ji obejde. To můžeme popsat například tak, že kostru zakořeníme, prohledáme ji do hloubky a zaznamenejme, jak jsme procházeli hranami. Každou hranou kostry přitom projdeme dvakrát – jednou dolů, podruhé nahoru. Tím však nedostaneme kružnici, nýbrž jen nějaký uzavřený sled, protože vrcholy navštívujeme vícekrát.

Sled tedy upravíme tak, že kdykoliv se dostává do již navštíveného vrcholu, přeskočí ho a přesune se až do nejbližšího dalšího nenavštíveného. Tak ze sledu vytvoříme hamiltonovskou kružnici a jelikož v grafu platí trojúhelníková nerovnost, celková délka nevzrostla. (Pořadí vrcholů na kružnici můžeme získat také tak, že během prohledávání budeme vypisovat vrcholy při jejich první návštěvě. Rozmyslete si, že je to totéž.)



Obrázek 19.10: Obchodní cestující obchází kostru

Věta: Nalezená kružnice není delší než dvojnásobek optima.

Důkaz: Označme T délku minimální kostry, A délku kružnice vydané naším algoritmem a O (optimum) délku nejkratší hamiltonovské kružnice. Z toho, jak jsme kružnici vytvořili,

víme, že $A \leq 2T$. Platí ovšem také $T \leq O$, jelikož z každé hamiltonovské kružnice vznikne vynecháním hrany kostra a ta nemůže být kratší než minimální kostra. Složením obou nerovností získáme $A \leq 2T \leq 2O$. $\square$

Sestrojili jsme 2-aproximační algoritmus pro problém obchodního cestujícího. Dodejme ještě, že trochu složitějším trikem lze tento problém 1.5-aproximovat a že v některých metrických prostorech (třeba v euklidovské rovině) lze v polynomiálním čase najít $(1 + \varepsilon)$ -aproximaci pro libovolné $\varepsilon > 0$. Ovšem čím menší je ε , tím déle algoritmus poběží.

Trojúhelníková nerovnost ovšem byla pro tento algoritmus klíčová. To není náhoda – hned dokážeme, že bez tohoto předpokladu je libovolná aproximace stejně těžká jako přesné řešení.

Věta: Pokud pro nějaké reálné $t \geq 1$ existuje polynomiální t -aproximační algoritmus pro problém obchodního cestujícího v úplném grafu (bez požadavku trojúhelníkové nerovnosti), pak je $P = NP$.

Důkaz: Ukážeme, že pomocí takového aproximačního algoritmu dokážeme v polynomiálním čase zjistit, zda v libovolném grafu existuje hamiltonovská kružnice, což je NP-úplný problém.

Dostali jsme graf G , ve kterém hledáme hamiltonovskou kružnici (zkráceně HK). Doplníme G na úplný graf G' . Všem původním hranám nastavíme délku na 1, těm novým na nějaké dost velké číslo c . Kolik to bude, určíme za chvíli.

Graf G' je úplný, takže v něm určitě nějaké HK existují. Ty, které se vyskytují i v původním grafu G , mají délku přesně n . Jakmile ale použijeme jedinou hranu, která z G nepochází, vzroste délka kružnice alespoň na $n - 1 + c$.

Podle délky nejkratší HK v G' tedy dokážeme rozpoznat, zda existuje HK v G . Potřebujeme to ovšem zjistit i přes zkreslení způsobené aproximací. Musí tedy platit $tn < n - 1 + c$. To snadno zajistíme volbou hodnoty c větší než $(t - 1)n + 1$.

Naše konstrukce přidala polynomiálně mnoho hran s polynomiálně velkým ohodnocením, takže graf G' je polynomiálně velký vzhledem ke G . Rozhodujeme tedy existenci HK v polynomiálním čase a $P = NP$. $\square$

Podobně můžeme dokázat, že pokud $P \neq NP$, neexistuje pro problém obchodního cestujícího ani pseudopolynomiální algoritmus. Stačí původním hranám přiřadit délku 1 a novým délku 2.

Aproximační schéma pro problém batohu

Již víme, jak optimalizační verzi problému batohu vyřešit v čase $\mathcal{O}(nC)$, pokud jsou hmotnosti i ceny na vstupu přirozená čísla a C je součet všech cen. Jak si poradit, pokud je C obrovské? Kdybychom měli štěstí a všechny ceny byly násobky nějakého čísla p , mohli bychom je tímto číslem vydělit. Tak bychom dostali zadání s menšími čísly, jehož řešením by byla stejná množina předmětů jako u zadání původního.

Když nám štěstí přát nebude, můžeme přesto zkusit ceny vydělit a výsledky nějak zaokrouhlit. Optimální řešení nové úlohy pak sice nemusí odpovídat optimálnímu řešení té původní, ale když nastavíme parametry správně, bude alespoň jeho dobrou aproximací. Budeme se snažit relativní chybu omezit libovolným $\varepsilon > 0$.

Základní myšlenka: Označíme $c_{\max}$ maximum z cen c_i . Zvolíme nějaké přirozené číslo $M < c_{\max}$ a zobrazíme interval cen $[0, c_{\max}]$ na $\{0, \dots, M\}$ (tedy každou cenu znásobíme poměrem $M/c_{\max}$ a zaokrouhlíme). Jak jsme tím zkreslili výsledek? Všimněme si, že efekt je stejný, jako kdybychom jednotlivé ceny zaokrouhlili na násobky čísla $c_{\max}/M$ (prvky z intervalu $[i \cdot c_{\max}/M, (i+1) \cdot c_{\max}/M)$ se zobrazí na stejný prvek). Každé c_i jsme tím tedy změnili o nejvýše $c_{\max}/M$, celkovou cenu libovolné podmnožiny předmětů pak nejvýše o $n \cdot c_{\max}/M$. Navíc odstraníme-li ze vstupu předměty, které se samy nevejdou do batohu, má optimální řešení původní úlohy cenu $c^* \geq c_{\max}$, takže chyba naší aproximace nepřesáhne $n \cdot c^*/M$. Má-li tato chyba být shora omezena $\varepsilon \cdot c^*$, musíme zvolit $M \geq n/\varepsilon$.

Na této myšlence „kvantování cen“ je založen následující algoritmus.

Algoritmus APROXIMACEBATOHU

1. Odstraníme ze vstupu všechny předměty těžší než H .
2. Spočítáme $c_{\max} = \max_i c_i$ a zvolíme $M = \lceil n/\varepsilon \rceil$.
3. Kvantujeme ceny: Pro $i = 1, \dots, n$ položíme $\hat{c}_i \leftarrow \lfloor c_i \cdot M/c_{\max} \rfloor$.
4. Vyřešíme dynamickým programováním problém batohu pro upravené ceny $\hat{c}_1, \dots, \hat{c}_n$ a původní hmotnosti i kapacitu batohu.
5. Vybereme stejné předměty, jaké použilo optimální řešení kvantovaného zadání.

Rozbor algoritmu: Kroky 1–3 a 5 jistě zvládneme v čase $\mathcal{O}(n)$. Krok 4 řeší problém batohu se součtem cen $\hat{C} \leq nM \in \mathcal{O}(n^2/\varepsilon)$, což stihne v čase $\mathcal{O}(n\hat{C}) = \mathcal{O}(n^3/\varepsilon)$. Zbývá dokázat, že výsledek našeho algoritmu má opravdu relativní chybu nejvýše ε .

Označme P množinu předmětů použitých v optimálním řešení původní úlohy a $c(P)$ cenu tohoto řešení. Podobně Q bude množina předmětů v optimálním řešení nakvantované úlohy a $\hat{c}(Q)$ jeho hodnota v nakvantovaných cenách. Potřebujeme odhadnout ohodnocení množiny Q v původních cenách, tedy $c(Q)$, a srovnat ho s $c(P)$.

Nejprve ukážeme, jakou cenu má optimální řešení P původní úlohy v nakvantovaných cenách:

$$\begin{aligned}\hat{c}(P) &= \sum_{i \in P} \hat{c}_i = \sum_{i \in P} \left\lfloor c_i \cdot \frac{M}{c_{\max}} \right\rfloor \geq \sum_{i \in P} \left(c_i \cdot \frac{M}{c_{\max}} - 1 \right) \geq \\ &\geq \left(\sum_{i \in P} c_i \cdot \frac{M}{c_{\max}} \right) - n = c(P) \cdot \frac{M}{c_{\max}} - n.\end{aligned}$$

Nyní naopak spočítejme, jak dopadne optimální řešení Q nakvantovaného problému při přepočtu na původní ceny (to je výsledek našeho algoritmu):

$$c(Q) = \sum_{i \in Q} c_i \geq \sum_{i \in Q} \hat{c}_i \cdot \frac{c_{\max}}{M} = \left(\sum_i \hat{c}_i \right) \cdot \frac{c_{\max}}{M} = \hat{c}(Q) \cdot \frac{c_{\max}}{M} \geq \hat{c}(P) \cdot \frac{c_{\max}}{M}.$$

Poslední nerovnost platí proto, že $\hat{c}(Q)$ je optimální řešení kvantované úlohy, zatímco $\hat{c}(P)$ je nějaké další řešení téže úlohy, které nemůže být lepší.⁽⁴⁾ Teď už stačí složit obě nerovnosti a dosadit za M :

$$\begin{aligned}c(Q) &\geq \left(\frac{c(P) \cdot M}{c_{\max}} - n \right) \cdot \frac{c_{\max}}{M} \geq c(P) - \frac{n \cdot c_{\max}}{n/\varepsilon} \geq c(P) - \varepsilon c_{\max} \geq \\ &\geq c(P) - \varepsilon c(P) = (1 - \varepsilon) \cdot c(P).\end{aligned}$$

Na přechodu mezi řádky jsme využili toho, že každý předmět se vejde do batohu, takže optimum musí být alespoň tak cenné jako nejčennější z předmětů.

Shrňme, co jsme dokázali:

Věta: Existuje algoritmus, který pro každé $\varepsilon > 0$ nalezne $(1 - \varepsilon)$ -*aproximaci* problému batohu s n předměty v čase $\mathcal{O}(n^3/\varepsilon)$.

Dodejme ještě, že algoritmům, které dovedou pro každé $\varepsilon > 0$ najít v polynomiálním čase $(1 - \varepsilon)$ -aproximaci optimálního řešení, říkáme *polynomiální aproximační schémata* (PTAS – Polynomial-Time Approximation Scheme). V našem případě je dokonce složitost polynomiální i v závislosti na $1/\varepsilon$, takže schéma je *plně polynomiální* (FPTAS – Fully Polynomial-Time Approximation Scheme).

⁽⁴⁾ Zde nás zachraňuje, že ačkoliv u obou úloh leží optimum obecně jinde, obě mají stejnou množinu *přípustných řešení*, tedy těch, která se vejdou do batohu. Kdybychom místo cen kvantovali hmotnosti, nebyla by to pravda a algoritmus by nefungoval.

Cvičení

1. *Problém MaxCut*: vrcholy zadaného grafu chceme rozdělit do dvou množin tak, aby mezi množinami vedlo co nejvíce hran. Jinými slovy chceme nalézt bipartitní podgraf s co nejvíce hranami. Rozhodovací verze tohoto problému je NP-úplná, optimalizační verzi zkuste v polynomiálním čase 2-aproximovat. [N39]
- 2.* V problému *MaxE3-SAT* dostaneme formuli v CNF, jejíž každá klauzule obsahuje právě 3 různé proměnné, a chceme nalézt ohodnocení proměnných, při němž je splněno co nejvíce klauzulí. Rozhodovací verze je NP-úplná. Ukažte, že náhodné ohodnocení proměnných splní v průměru $7/8$ klauzulí. Z toho odvoďte deterministickou $7/8$ -aproximaci v polynomiálním čase. [N88]
3. *Hledové pokrytí*: Hledejme vrcholové pokrytí následujícím hledovým algoritmem. V každém kroku vybereme vrchol nejvyššího stupně, přidáme ho do pokrytí a odstraníme ho z grafu i se všemi již pokrytými hranami. Je nalezené pokrytí nejmenší? Nebo alespoň $\mathcal{O}(1)$ -aproximace nejmenšího?
- 4.* *Pokrytí jinak*: Uvažujme následující algoritmus pro nejmenší vrcholové pokrytí grafu. Graf projdeme do hloubky, do výstupu vložíme všechny vrcholy vzniklého DFS stromu kromě listů. Dokažte, že vznikne vrcholové pokrytí a že 2-aproximuje to nejmenší. [N121]
- 5.* V daném orientovaném grafu hledáme acyklický podgraf s co nejvíce hranami. Navrhnete polynomiální 2-aproximační algoritmus. [N30]
- 6.* *Obchodní cestující*: Řešení problému obchodního cestujícího hrubou silou by prohledávalo graf do hloubky a zkoušelo všechny hamiltonovské kružnice. To může v grafu na n vrcholech trvat až $n!$ kroků. Pokuste se najít rychlejší algoritmus. Dynamic-kým programováním lze dosáhnout složitosti $\mathcal{O}(2^n \cdot n^k)$ pro konstantní k . To je sice exponenciální, ale stále mnohem lepší než faktoriál. [N106]
7. *Konvexní obchodní cestující* chce navštívit všech n vrcholů zadaného konvexního n -úhelníku v rovině. Vzdálenosti měříme v běžném euklidovském smyslu. Najděte polynomiální algoritmus. [N96]

Dodatek A

Základy teorie grafů

A Základy teorie grafů

Jedním ze základních stavebních kamenů algoritmů v naší knize je matematická struktura, které se říká graf. Jelikož teorie grafů zatím není běžnou součástí školní matematiky, připojujeme tento dodatek. Klade si za cíl seznámit čtenáře se základními definicemi a nejdůležitějšími větami o grafech, které souvisí s tématy probíranými v Průvodci. Základní grafové algoritmy potom čtenář najde v kapitolách 5 až 7.

Na rozdíl od hlavní části knihy je dodatek napsaný zhuštěnou formou a není zamýšlen jako učebnice, spíš jako přehledový text. Jako vhodnou učebnici doporučujeme vynikající Kapitoly z diskrétní matematiky od Jiřího Matouška a Jaroslava Nešetřila [9].

A.1 Základní pojmy a vlastnosti

Definice: *Neorientovaný graf* je uspořádaná dvojice (V, E) , kde V je neprázdná konečná množina *vrcholů* (někdy též *uzlů*) a $E \subseteq \binom{V}{2}$ je množina *hran*.

Hrana je dvouprvková podmnožina V (čili neuspořádaná dvojice vrcholů). Značíme ji $\{u, v\}$, někdy také krátce uv . Symbolem $\binom{V}{2}$ zde rozumíme množinu všech dvojic vrcholů.

Nechť $e = \{u, v\}$ je hrana grafu. Pak řekneme, že vrcholy u a v jsou *koncové vrcholy* hrany e , vrchol u je *sousedem* v (a naopak) a vrcholy u a v jsou *incidentní* s hranou e .

Množinu vrcholů a hran grafu G budeme značit $V(G)$ a $E(G)$. Pro počet jeho vrcholů a hran se nejčastěji používají písmena $n = |V(G)|$ a $m = |E(G)|$.

Pro graf $G = (V, E)$ a jeho vrchol $v \in V$ značíme symbolem $\deg_G(v)$ počet hran grafu G incidentních s vrcholem v . Toto číslo nazveme *stupněm* vrcholu v v grafu G . Vrchol stupně 0 (tedy bez sousedů) nazveme *izolovaný*.

Symbolem $N_G(v)$ označíme množinu všech sousedů vrcholu v v grafu G . Tuto množinu nazveme (*otevřené*) *okolí* vrcholu v v grafu G . Platí tedy $\deg_G(v) = |N_G(v)|$. Tam, kde je z kontextu jasné, o který graf jde, píšeme prostě $\deg(v)$ a $N(v)$.

Graf je *r -regulární*, pokud každý jeho vrchol má stupeň r . Graf je *regulární*, pokud je r -regulární pro nějaké r .

Možnosti, jak grafy reprezentovat v paměti počítače, blíže rozebíráme v oddílu 5.3.

Základní druhy grafů

Definice: Pro $m \geq 0$ a $n \geq 1$ definujeme následující grafy:

- *Úplný graf* $K_n = (V, \binom{V}{2})$, kde $V = \{1, \dots, n\}$. Každé dva jeho vrcholy jsou sousední.